



UNIVERSIDAD TECMILENIO

Actividad 4

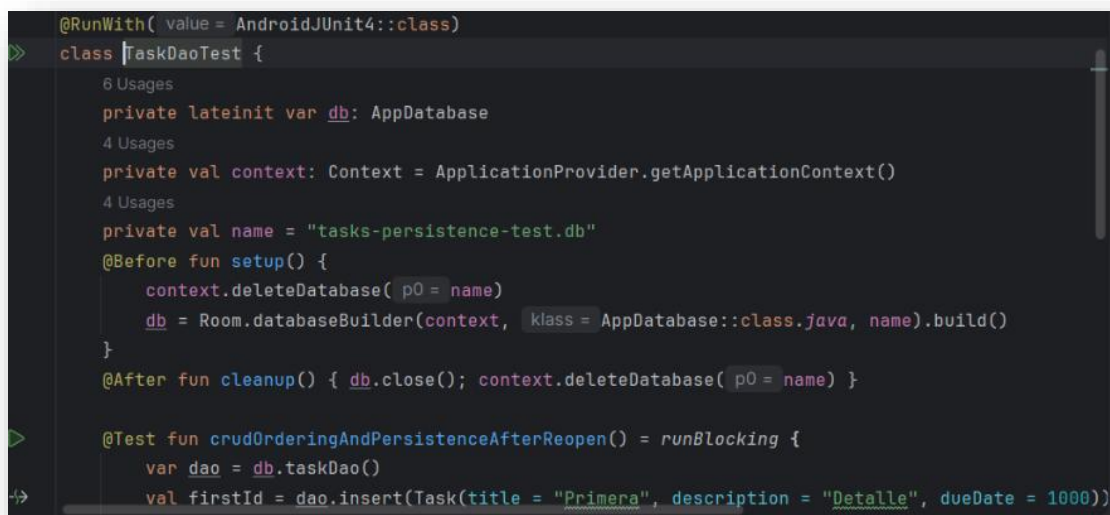


Nombre: Luis Fernando Núñez Díaz - Ricardo Baranda Cisneros – Bryan David Mariñelarena Ponce

Maestro: Adalberto Emmanuel Rojas Perea

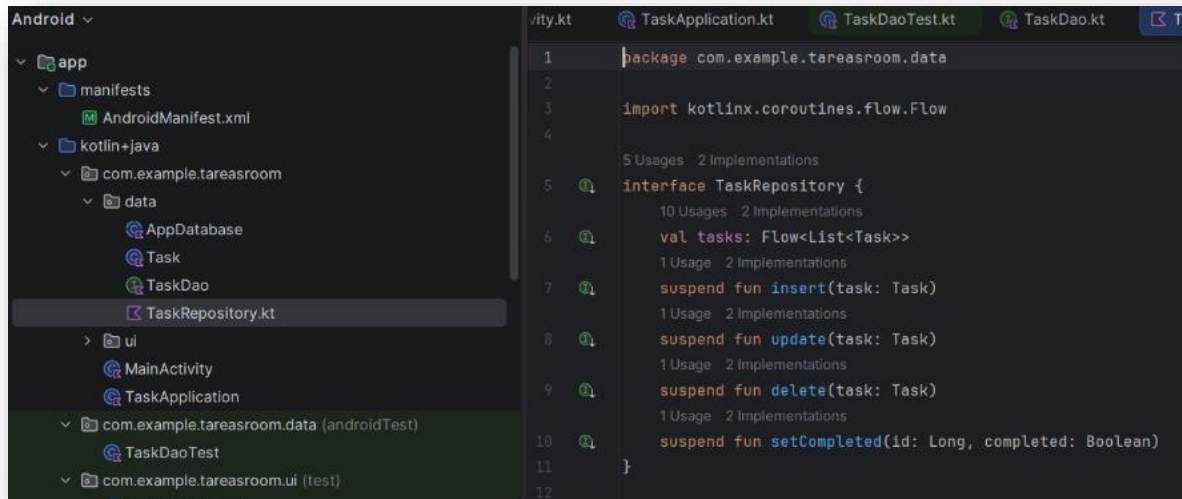
Materia: Desarrollo de Aplicaciones Móviles

Equipo: Pingüino



3. Configuración de la base de datos:

- Implementa una clase AppDatabase abstracta con patrón Singleton.
- Define un repositorio (TaskRepository) que use el DAO para operaciones.



The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes a package `com.example.tareasroom` with sub-packages `data` and `ui`. The `data` package contains `AppDatabase`, `Task`, `TaskDao`, and `TaskRepository.kt`. The `ui` package contains `MainActivity` and `TaskApplication`. The code editor shows the `TaskRepository` interface in `TaskRepository.kt`:

```
1 package com.example.tareasroom.data
2
3 import kotlinx.coroutines.flow.Flow
4
5 5 Usages 2 Implementations
6 interface TaskRepository {
7     10 Usages 2 Implementations
8     val tasks: Flow<List<Task>>
9     1 Usage 2 Implementations
10    suspend fun insert(task: Task)
11    1 Usage 2 Implementations
12    suspend fun update(task: Task)
13    1 Usage 2 Implementations
14    suspend fun delete(task: Task)
15    1 Usage 2 Implementations
16    suspend fun setCompleted(id: Long, completed: Boolean)
17 }
```

4. Integración con ViewModel:

- Crea un TaskViewModel que exponga los datos mediante StateFlow y maneje eventos como addTask o deleteTask.
- Usa viewModelScope para operaciones asíncronas.



The screenshot shows the `MainActivity` class in the `ui` package. The code is as follows:

```
</> class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TaskTheme {
                val model: TaskViewModel = viewModel(factory = TaskViewModel.Factory)
                TaskListScreen( viewModel = model)
            }
        }
    }
}
```

5. Diseño de UI:

- Implementa una pantalla con:
- LazyColumn para mostrar tareas.
- FloatingActionButton para abrir un diálogo de creación.
- Switch para marcar tareas como completadas.



Actividad 4

Descripción

Desarrollar una aplicación móvil que combine animaciones fluidas, gestos intuitivos y almacenamiento local con Room, priorizando una experiencia de usuario cohesiva y eficiente.

Instrucciones

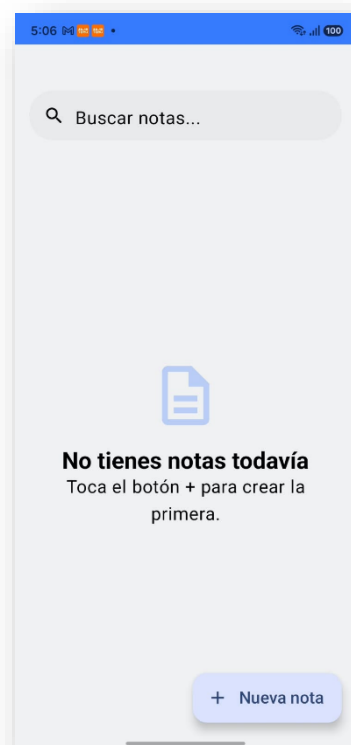
1. Configuración inicial:

Para el desarrollo de la aplicación se creó un proyecto en **Android Studio** utilizando **Kotlin** y **Jetpack Compose**. Para almacenar la información de las notas se utilizó **Room**, que permite trabajar con una base de datos local SQLite.

La aplicación utiliza una entidad llamada **Note**, encargada de representar cada nota almacenada. Esta contiene los siguientes datos:

- **id:** identificador único de la nota.
- **title:** título de la nota.
- **content:** contenido.
- **date:** fecha de creación o modificación.

También se implementaron los componentes de Room necesarios, como **NoteDao**, **NoteDatabase**, los conversores de datos y el repositorio.



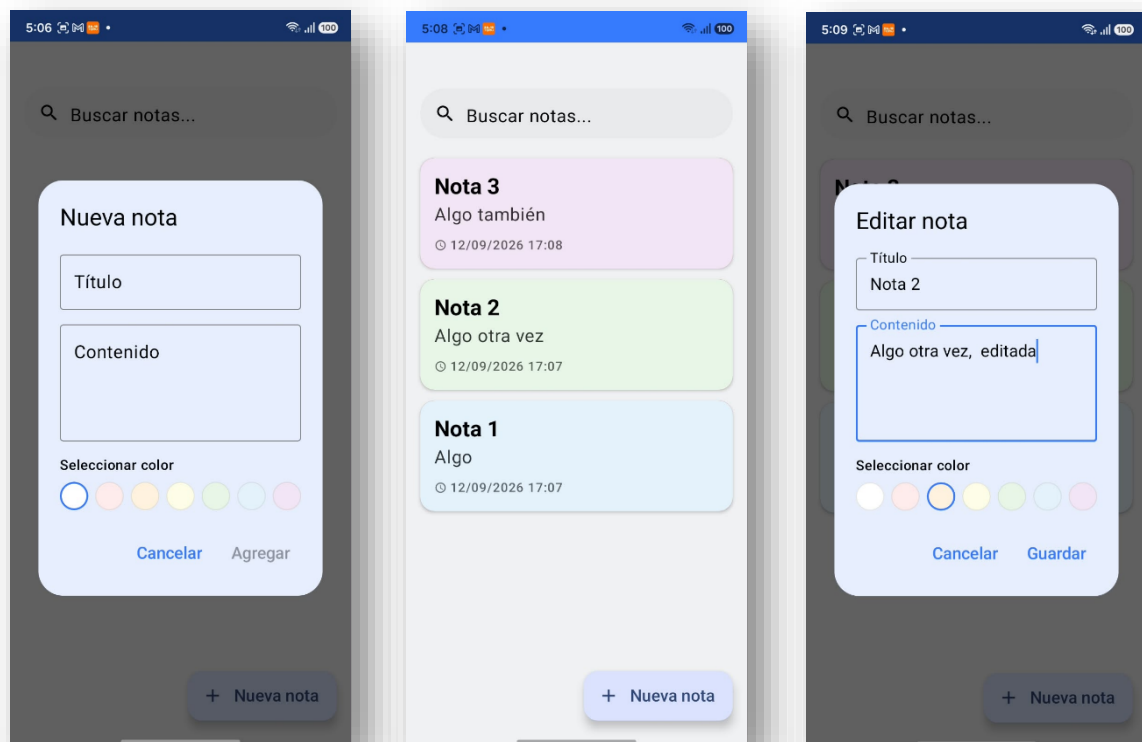
2. Implementar operaciones CRUD:

Se implementaron las operaciones **CRUD** para permitir administrar las notas dentro de la aplicación:

- **Crear:** permite agregar una nueva nota.
- **Leer:** muestra las notas almacenadas en la base de datos.
- **Actualizar:** permite modificar una nota existente.
- **Eliminar:** permite eliminar una nota.

Para gestionar estas operaciones se utilizó **RoomDatabase**, un Repository y un ViewModel. El NoteDao se encarga de realizar las operaciones directamente sobre la base de datos.

La información se obtiene mediante Flow, permitiendo que los cambios realizados en la base de datos se reflejen automáticamente en la interfaz.

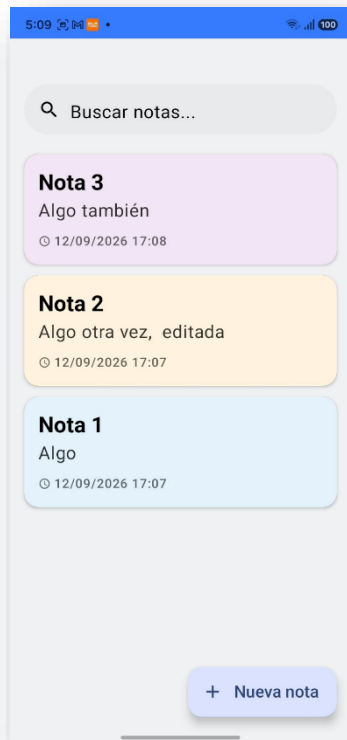


3. Diseñar interfaz con animaciones:

La interfaz fue desarrollada utilizando **Jetpack Compose** y **Material 3**.

Se implementó una animación para que las notas aparezcan de manera visual al agregarse. Para esto se utilizó `AnimatedVisibility`, combinando animaciones de desplazamiento horizontal y aparición gradual mediante `slideInHorizontally` y `fadeIn`.

Esto permite que la aplicación tenga una interacción más dinámica al momento de agregar una nota.



4. Añadir gestos:

Se implementó un gesto de **deslizamiento (swipe)** para eliminar las notas.

El usuario puede deslizar una nota hacia un lado para eliminarla. En la versión implementada se utiliza `SwipeToDismissBox`, que permite realizar esta acción de manera sencilla dentro de Jetpack Compose.

Esta función facilita la interacción del usuario y evita tener que agregar un botón adicional para eliminar cada nota.



5. Pruebas y ajustes:

Finalmente, se realizaron pruebas para comprobar el correcto funcionamiento de la aplicación.

Se verificó principalmente:

- Creación de nuevas notas.
- Visualización de las notas almacenadas.
- Edición de notas.
- Eliminación mediante el gesto de swipe.
- Persistencia de los datos después de cerrar y volver a abrir la aplicación.
- Funcionamiento en emulador o dispositivo Android.

También se implementaron pruebas unitarias e instrumentadas para comprobar el funcionamiento de los conversores y las operaciones de la base de datos.

Además, se utilizó una herramienta de inteligencia artificial como apoyo durante el desarrollo para revisar código, resolver errores y verificar posibles soluciones.

